【网络诊断修复工具】

帐号 用户名/Email 自动登录 找回密码
密码 登录 注册[Register]
用QQ帐号登录
只需一步，快速开始

网站 新帖 专辑 总版规 爱盘 帮助 关注微信 快捷导航

请输入搜索内容 搜索 热搜: ctf 新手 脱壳 教程

查看: 13111 | 回复: 178

wshuo

[其他原创] 无源NFC墨水屏制作 [复制链接]

发表于 2026-1-6 02:20 楼主 电梯直达

优秀

本帖最后由 wshuo 于 2026-1-7 14:21 编辑

1.前言

之前在网上看到一个开源项目：https://oshwhub.com/ludas/nfc-epd-driver 觉得很有意思，想复刻一下，但是复刻失败了，因为我不清楚其中原理，也没有源码可供调试，所有花时间好好研究了一下。

无源NFC墨水屏的意思就是不需要供电或者内置电池，利用NFC的感应磁场来进行供电，然后通过程序控制对墨水屏幕进行刷新，得益于墨水屏掉电依然可以显示内容的特性，就可以实现一个完全不用电源并且可以被刷新的屏幕了。

先放成品
视频演示：https://www.bilibili.com/video/BV1rWitBTEdJ/

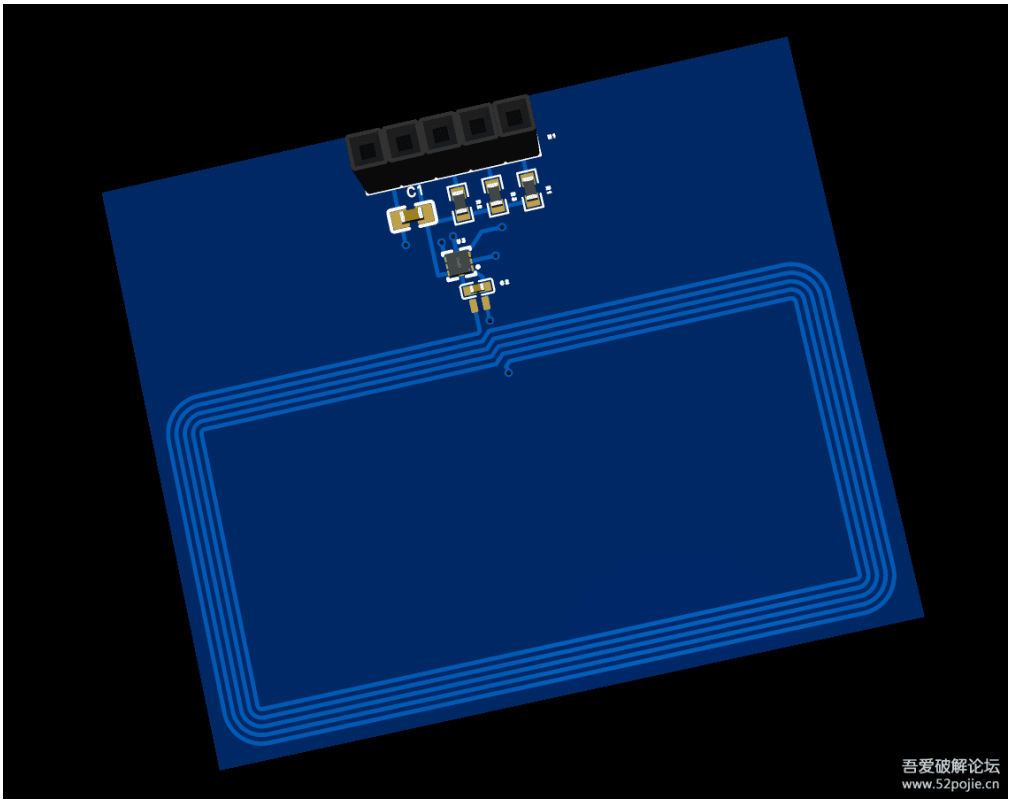
2. 分析

原项目采用 nxp芯片 NT3H1101W0FHKH， 这个芯片具有能量收集的功能（感应到NFC场，输出3v3），主控mcu 使用的是低功耗 意法半导体的 STM32L011D4P6。
在刷新墨水屏屏幕时， NT3H1101W0FHKH 接收到数据，并给主控mcu供电，通过 I2C协议 与主控mcu传递数据，然后主控mcu在此同时对墨水屏 进行刷新。

所有首先我要搞明白 NT3H1101W0FHKH 这颗芯片的使用

3. NT3H1101

这里我简单画了个测试版：



板子很简单，I2C总线和FD引脚上加了上拉电阻，然后加了去耦电容，还有一个谐振匹配的电容（这个加不加都能识别到）。
关键在于引出所有 NT3H1101 引脚 （I2C通信引脚，FD场检测引脚，能量回收供电引脚），这样方便我后续测试。

与我熟悉的 主控mcu进行数据交互，通过简单的I2C地址扫描，找到了 NT3H1101 I2C通信地址：

复制代码 隐藏代码

```
void setup() {
  Wire.begin();
  Serial.begin(9600);
  Serial.println("\nI2C Scanner");
}
void loop() {
  byte error, address;
  int nDevices;
  Serial.println("Scanning...");
  nDevices = 0;
  for(address = 1; address < 127; address++ ) {
    Wire.beginTransmission(address);
    error = Wire.endTransmission();
    if (error == 0) {
      Serial.print("I2C device found at address 0x");
      if (address<16) Serial.print("0");
      Serial.println(address, HEX);
      nDevices++;
    }
  }
  if (nDevices == 0) Serial.println("No I2C devices found
  delay(5000);
}
```

那么剩下的主控MCU 与 NT3H1101 进行数据交互，通过阅读数据手册，得知，NT3H1101 与 mcu进行数据交互时，进入了 passthrough模式， 这个模式前提是需要 NT3H1101 有外部供电，一旦掉电后 就会退出 passthrough模式（这里可以用 NT3H1101自己回收的能量给自己供电），除此还需要对 NT3H1101 会话寄存器进行一些设置才能开启passthrough模式。

另外这里说一下 俩个概念：setting register 和 session register，EEPROM
setting register 简单可以理解成掉电不丢失，在每次por后数据依然存在。
session register 简单可以理解成掉电丢失，每次por后丢失，记录芯片的一些状态信息及临时配置。
EEPROM 是掉电不丢失的，有读写寿命，就像对NFC卡片写入个URL，或者打开APP这类数据都写入到这部分空间，本次不会用到这部分空间。

每一次上电，session register 的一部分配置选项会从 setting register 中载入，而我们要修改的 passthrough模式，只存在 session register中的（因为只有存在

供电的情况才会使用到passthrough），所以每一次上电后主控mcu都需要通过I2C总线对 NT3H1101 的passthrough模式进行设置，才可以进行数据传输。

Table 6. NTAG I²C 1k memory organization from the I²C interface

I²C block address		Byte number within a block				Access conditions
		0	1	2	3	
		4	5	6	7	
		8	9	10	11	
Dec.	Hex.	12	13	14	15	
0	00h	I²C addr.*	Serial number			R&W/READ
		Serial number			Internal	READ
		Internal		Static lock bytes		READ/R&W
		Capability Container (CC)				READ&WRITE
1	01h	User memory				READ&WRITE
...	...					
55	37h					
56	38h	User memory				READ&WRITE
		User memory				READ&WRITE
		Dynamic lock bytes			00h	READ&WRITE
		00h	00h	00h	00h	READ
57	39h	Invalid access - returns NAK				n.a.
58	3Ah	Configuration registers				see 8.3.11
		00h	00h	00h	00h	READ
		00h	00h	00h	00h	
59	3Bh	Invalid access - returns NAK				n.a.
...	...					
247	F7h					
248	F8h	SRAM memory (64 bytes)				READ&WRITE
...	...					
251	FBh					
...	...	Invalid access - returns NAK				n.a.
254	FEh	Session registers (requires READ register command)				see 8.3.11
		00h	00h	00h	00h	READ
		00h	00h	00h	00h	
...	...	Invalid access - returns NAK				n.a.

Remark: * The byte 0 of block 0 is always read as 04h. Writing to this byte modifies the I²C address.

要修改session register 在 NT3H1101 中 I2C的访问地址是0xFE。
在I2C接口下，一个地址都是对应16字节数据，只不过在session register中只有前7个字节才有用，每个字节都对应一个寄存器名字：

Table 12. Session registers NTAG I²C 1k and 2k

RF address (sector 3)		I²C Address		Byte number			
Dec	Hex	Dec	Hex	0	1	2	3
248	F8h	254	FEh	NC_REG	LAST_NDEF_BLOCK	SRAM_MIRROR _BLOCK	WDT_LS
249	F9h			WDT_MS	I2C_CLOCK_STR	NS_REG	00h fixed

相当于一个寄存器占了一个字节，一个字节中的8位，每一位（Filed）对应不同功能，比如我要修改的passthrough模式，是在NC_REG寄存器中：

Table 14. Session register bytes

Bit	Field	Access via RF	Access via I²C	Default values	Description
Session register: NC_REG					
7	I2C_RST_ON_OFF	READ	R&W	-	see configuration bytes description
6	PTHRU_ON_OFF	READ	R&W	0b	1b… enables data transfer via the SRAM buffer (Pass-through mode)
5	FD_OFF	READ	R&W	-	see configuration bytes description
4					
3	FD_ON	READ	R&W		
2					
1	SRAM_MIRROR_ ON_OFF	READ	R&W	0b	1b enables SRAM mirroring
0	PTHRU_DIR	READ	R&W		see configuration bytes description
Session register: LAST_NDEF_BLOCK					
7-0	LAST_NDEF_ BLOCK	READ	R&W	-	see configuration bytes description
Session register: SRAM_MIRROR_BLOCK					
7-0	SRAM_MIRROR_ BLOCK	READ	R&W	-	see configuration bytes description
Session register: WDT_LS					

他在NC_REG寄存器的第六位(PTHRU_ON_OFF)，代码实现对第六位寄存器写操作：

复制代码 隐藏代码

```
void startPassthrough() {
  Wire.beginTransmission(NT3H_I2C_ADDR);
  Wire.write(0xFE);  //会话寄存器的 起始地址
  Wire.write(0x00);  //第一个session register 即 NC_REG
  Wire.write(0x40); //发送掩码，告诉要修改这个寄存器的第几位
  Wire.write(0x40);   // 将其对应位修改为1
  Wire.endTransmission();
}
```

这里掩码操作是必须的，这个是NT3H1101 寄存器写入的规则，防止在喂入一个字节数据的时候修改到其他位。

开启了passthrogh模式后，我们需要知道RF接口到 I2C接口数据写入到了哪个地址，以及主控MCU什么时间开始读取对应地址的数据：

11.3.2 RF to I²C Data transfer

If the RF interface is enabled (RF_LOCKED = 1b) and data is written to the terminator block/page of the SRAM via the RF interface, at the end of the WRITE command, bit SRAM_I2C_READY is set to 1b and bit RF_LOCKED is set to 0b automatically, and the NTAG I²C is locked to the I²C interface.

To signal to the host that data is ready to be read following mechanisms are in place:

- The host polls/reads bit SRAM_I2C_READY from NS_REG (see Table 14) to know if data is ready in SRAM
- A trigger on the FD pin indicates to the host that data is ready to be read from SRAM. This feature can be enabled by programming bits 5:2 (FD_OFF, FD_ON) of the NC_REG appropriately (see Table 13)

这里提供了俩种方式：

1. 轮询 NS_REG寄存器的 SRAM_I2C_READY位。
   2.利用FD引脚，在settings register中配置功能（可以配置场出现，数据到达等事件改变FD引脚电平状态），然后主控mcu使用中断机制来获悉数据到达。

这里我采用 轮询寄存器方式，那么需要对寄存器进行读取：

Session register: NS_REG					
7	NDEF_DATA_READ	READ	READ	0b	1b… all data bytes read from the address specified in LAST_NDEF_BLOCK. value is reset to 0b when read
6	I2C_LOCKED	READ	R&W	0b	1b… Memory access is locked to the I²C interface
5	RF_LOCKED	READ	READ	0b	1b… Memory access is locked to the RF interface
4	SRAM_I2C_READY	READ	READ	0b	1b… data is ready in SRAM buffer to be read by I2C
3	SRAM_RF_READY	READ	READ	0b	1b… data is ready in SRAM buffer to be read by RF
2	EEPROM_WR_ERR	READ	R&W	0b	1b… HV voltage error during EEPROM write or erase cycle Needs to be written back via I²C to 0b to be cleared
1	EEPROM_WR_BUSY	READ	READ	0b	1b… EEPROM write cycle in progress - access to EEPROM disabled 0b… EEPROM access possible
0	RF_FIELD_PRESENT	READ	READ	0b	1b… RF field is detected

复制代码 隐藏代码

```
boolean checkReady(){
    byte value;
    Wire.beginTransmission(NT3H_I2C_ADDR);
    Wire.write(0xFE); //会话寄存器的 起始地址
    Wire.write(0x06); //选中 NC_REG寄存器
    Wire.endTransmission();
    Wire.requestFrom(NT3H_I2C_ADDR,1);
    if (Wire.available() == 1) {
      value=Wire.read();
    }
    return value & 0b10000; // 判断NS_REG寄存器的第4位，即SRA
}
```

OK，到此俩个寄存器读写已经搞好了，剩下就是读取一个地址，将RF-> I2C的数据读取出来，NT3H1101 在有外部供电的情况下，SRAM空间会映射到I2C地址 0xF8 - 0xFB 页，每个页有16个字节，所有SRAM一共有 64字节。

复制代码 隐藏代码

```
void ReadDataBlock(const byte block_address, uint8_t *out
{
      Wire.beginTransmission(NT3H_I2C_ADDR);
      Wire.write(block_address);
      Wire.endTransmission();
      Wire.requestFrom(NT3H_I2C_ADDR, out_buffer_length);
      if (Wire.available() == out_buffer_length) {
        for (int i = 0; i < out_buffer_length; i++)
        {
          out_buffer[i+16*(block_address-0xf8)] = Wire.re
        }
      }
}
```

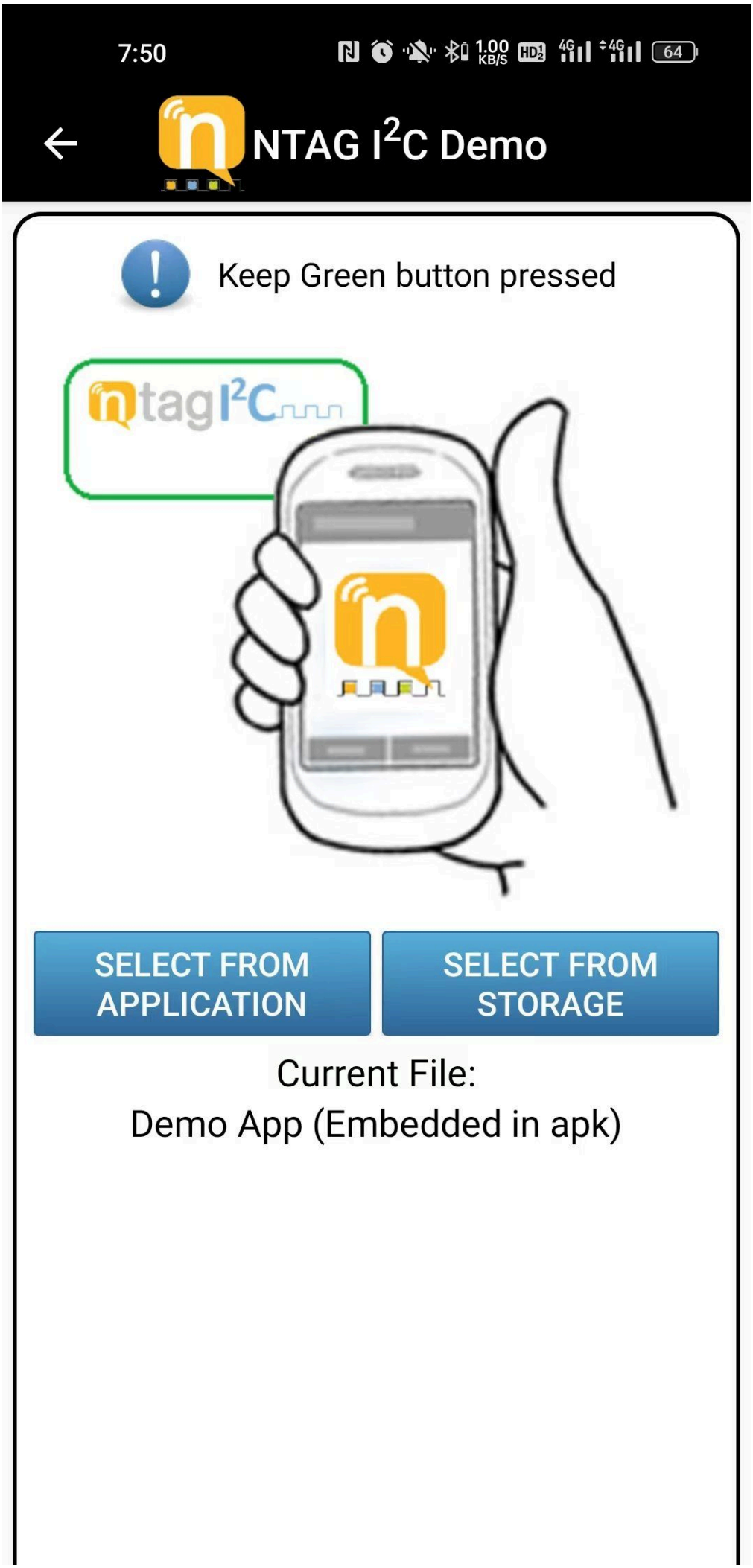
```
void readPages(uint8_t startPage, uint8_t endPage, uint8_
    for (uint8_t page = startPage; page <= endPage; page+
        ReadDataBlock(page, data64,16)
    }
}

void loop() {
        startPassthrough();
        if (checkReady()){
          byte data64[64] = {0};
          readPages(0xf8,0xfb, data64);
          for (size_t i = 0; i < 64; i++)
          {
            printHex(data64[i]);
          }
        }
}
```

代码很简单，每次将从RF接口读取的数据打印出来。
到此，基本的通信RF -> I2C 数据传递算是已经通过了。

4. APP端的大数据传输失败问题

这里说一下我手机上用的APP是：**NXP I2C Demo**


7:50
NTAG I2C Demo
Keep Green button pressed
ntag I2C
SELECT FROM APPLICATION
SELECT FROM STORAGE
Current File:
Demo App (Embedded in apk)

这个APP提供了NXP官方提供开发板的一些demo 设置，以及对NDFF的读写 (前面提到的对EEPROM的读写)，这里我主要利用对固件烧录功能，这部分功能就是通过passthrough模式进行 RF-> I2C数据传输，测试程序也是通过这部分功能进行测试的。

但在实际测试过程中我发现对大文件的传输，必然失败，并且是处于一个固定大小。很明显这种不是偶发问题，偶发问题不会在固定大小是失败。

经过排查了logcat 日志：

复制代码 隐藏代码

```
$ adb logcat |grep FLASH
01-05 20:19:51.661 32438 20705 D FLASH   : Flashing to st
01-05 20:19:51.899 32438 20705 D FLASH   : Start Block wr
01-05 20:19:51.999 32438 20705 D FLASH   : Starting Block
01-05 20:20:20.316 32438 20705 D FLASH   : All Blocks wri
01-05 20:20:21.317 32438 20705 D FLASH   : Wait finished
01-05 20:20:21.353 32438 20705 D FLASH   : Block read
01-05 20:20:21.353 32438 20705 D FLASH   : was nak
```

通过日志看到，这个app对于大文件进行了切片，最后到 was nak就会fail, 所有我找了一下这个app的源码，研究一下这个app内部逻辑，好在找到nxp官方提供了app 源码包（省下我逆向app的时间了），虽然版本是低版本，但是逻辑没变化，寻找关键日志：

复制代码 隐藏代码

```
wshuo@wshuo-desktop:~/Downloads/SW3648/NTAG_I2C_Demo_Andr
grep: ./build/intermediates/transforms/dex/debug/folders/
grep: ./build/intermediates/classes/debug/com/nxp/nfc_dem
./src/com/nxp/nfc_demo/reader/Ntag_I2C_Demo.java:
```

Ntag_I2C_Demo.java:

复制代码 隐藏代码

```
    public Boolean Flash(byte[] bytesToFlash) {
        int sectorSize = PAGE_SIZE;  //4096

        byte[] data = null;
        byte[] flashData = null;

        try {
            int length = bytesToFlash.length;  //总长度
```

```
int flashes = length / sectorSize + (length %
int blocks = (int) Math.ceil(length / (float)

// Set the number of writings
FlashMemoryActivity.setFLashDialogMax(blocks)

for (int i = 0; i < flashes; i++) {
    int flash_addr = 0x4000 + i * sectorSize;
    int flash_length = 0;

    if (length - (i + 1) * sectorSize < 0) {
        flash_length = roundUp(length % secto
        flashData = new byte[flash_length];
        Arrays.fill(flashData, (byte) 0);
        System.arraycopy(bytesToFlash, i * se
    } else {
        flash_length = sectorSize;
        flashData = new byte[flash_length];
        System.arraycopy(bytesToFlash, i * se
    }

    data = new byte[reader.getSRAMSize()];
    data[reader.getSRAMSize() - 4] = 'F';
    data[reader.getSRAMSize() - 3] = 'P';

    data[reader.getSRAMSize() - 8] = (byte) (
    data[reader.getSRAMSize() - 7] = (byte) (
    data[reader.getSRAMSize() - 6] = (byte) (
    data[reader.getSRAMSize() - 5] = (byte) (

    data[reader.getSRAMSize() - 12] = (byte)
    data[reader.getSRAMSize() - 11] = (byte)
    data[reader.getSRAMSize() - 10] = (byte)
    data[reader.getSRAMSize() - 9] = (byte) (

    Log.d("FLASH", "Flashing to start");
    reader.writeSRAMBlock(data, null);
    Log.d("FLASH", "Start Block write " + (i

    reader.waitforI2Cread(100);

    Log.d("FLASH", "Starting Block writing");
    reader.writeSRAM(flashData, R_W_Methods.F
    Log.d("FLASH", "All Blocks written");

    reader.waitforI2Cwrite(500);
    Thread.sleep(500);

    Log.d("FLASH", "Wait finished");
    byte[] response = reader.readSRAMBlock();
    Log.d("FLASH", "Block read");
```

```
            if (response[reader.getSRAMSize() - 4] !=
                Log.d("FLASH", "was nak");
                return false;
            }
            Log.d("FLASH", "was ack");
        }
        Log.d("FLASH", "Flash completed");

        data = new byte[reader.getSRAMSize()];
        data[reader.getSRAMSize() - 4] = 'F';
        data[reader.getSRAMSize() - 3] = 'S';
        reader.writeSRAMBlock(data, null);

        // Wait for the I2C to be ready
        reader.waitforI2Cread(DELAY_TIME);
        return true;
```

可以看到这个app在发送文件开始会发送一个启始头：这个头的 flash_length 为烧录固件长度，flash_addr 为烧录地址（这部分信息对我们这个项目没用），每一切片结束后需要I2C给 RF一个 ACK 表示，才会继续下一个切片。
所有这里我需要解决俩个问题：

1. 剔除无效数据，flash_length flash_addr数据，方案：利用关键标识符号 FP 以及 前 28 字节都为数据0（如果所有数据当做有效数据会对墨水屏显示造成干扰）。
2. 在每一切片结束后，构造 ACK 数据返回给 APP。

关于第一点很好解决，简单逻辑判断即可，我在第二点问题上解决了很久，原因就在于 RF在往里写数据，I2C也在往里写数据，这其中有个同步的问题。

我尝试过一直通过I2C写入ACK数据，发现只有在I2C读取完整个SRAM空间前写入有效，但是这样就破坏了RF写入的数据。
无奈只能寻找NXP官方单片机的代码参考（看ACK是什么时机写入的）：

复制代码 隐藏代码

```
wshuo@wshuo-desktop:~/Downloads/SW3647/workspace_ntag_i2c
./NTAG_I2C_Explorer_BootLoader/src/main.c:      sram_buf[
./NTAG_I2C_Explorer_BootLoader/src/main.c:      sram_buf[
./NTAG_I2C_Explorer_BootLoader/src/hid_desc.c:  'A', 0,
wshuo@wshuo-desktop:~/Downloads/SW3647/workspace_ntag_i2c
```

我发现一条比较重要的信息：

复制代码 隐藏代码

```
    if (flash((void*) addresse, (void*) data, size)) {
        HW_switchLEDs(REDLED);
        HAL_Timer_delay_ms(10);
        HW_switchLEDs(LEDOFF);

        sram_buf[NFC_MEM_SRAM_SIZE - 4] = 'N';
        sram_buf[NFC_MEM_SRAM_SIZE - 3] = 'A';
        sram_buf[NFC_MEM_SRAM_SIZE - 2] = 'K';
    } else {
        sram_buf[NFC_MEM_SRAM_SIZE - 4] = 'A';
        sram_buf[NFC_MEM_SRAM_SIZE - 3] = 'C';
        sram_buf[NFC_MEM_SRAM_SIZE - 2] = 'K';
    }

    NFC_SetTransferDir(ntag_handle, I2C_TO_RF);
    NFC_SetPthruOnOff(ntag_handle, TRUE);

    // write back Data
    NFC_WriteBytes(ntag_handle, NFC_MEM_ADDR_START_SRAM,
            NFC_MEM_SRAM_SIZE);
```

其在I2C写入数据之前，设置了 NC_REG寄存器的 PTHRU_DIR 位。其是NC_REG的第0位， 将这一步代码加入后，果然没问题了。

完整测试代码：

复制代码 隐藏代码

```
#include <Wire.h>
#include <Arduino.h>

#define NT3H_I2C_ADDR 0x55
#define SRAM_START_ADDR 0xf8
#define SRAM_END_ADDR   0xFB

#define NT3H1101_NC_REG      0x00
#define NT3H1101_SESSION_REG 0xFE

#define SRAM_SIZE 64

boolean flag = true;
uint32_t haveWriten = 0;
uint32_t allCount = 0;
bool stopFlag = false;

void writeRegister(uint8_t reg, uint8_t value) {
  Wire.beginTransmission(NT3H_I2C_ADDR);
  Wire.write(0xfe);
  Wire.write(0x00);
```

```
  Wire.write(reg);
  Wire.write(value);
  uint8_t error = Wire.endTransmission();

  if (error == 0) {
    Serial.print("Write successful to register 0x");
    Serial.println(reg, HEX);
  } else {
    Serial.print("Error writing to register 0x");
    Serial.print(reg, HEX);
    Serial.print(": Error code ");
    Serial.println(error);
  }
}

void setI2CtoNFC(){
  stopPassthrough();
  Wire.beginTransmission(NT3H_I2C_ADDR);
  Wire.write(0xfe);
  Wire.write(0x00);
  Wire.write(0x1);
  Wire.write(0x0);
      uint8_t error = Wire.endTransmission();

    if (error == 0) {
      Serial.println("setI2CtoNFC success");
    } else {
      Serial.println("setI2CtoNFC fail");
  }
  startPassthrough();
}

void setNFCtoI2C(){
  //passthrough and NFCtoI2c
  stopPassthrough();

  Wire.beginTransmission(NT3H_I2C_ADDR);
  Wire.write(0xfe);
  Wire.write(0x00);
  Wire.write(0x1);
  Wire.write(0x1);
      uint8_t error = Wire.endTransmission();

    if (error == 0) {
      Serial.println("setI2CtoNFC success");
    } else {
      Serial.println("setI2CtoNFC fail");
  }
  startPassthrough();
}
```

```
void startPassthrough() {
  Wire.beginTransmission(NT3H_I2C_ADDR);
  Wire.write(0xfe);
  Wire.write(0x00);
  Wire.write(0x40);
  Wire.write(0x40);
  Wire.endTransmission();
}

void stopPassthrough() {
  Wire.beginTransmission(NT3H_I2C_ADDR);
  Wire.write(0xfe);
  Wire.write(0x00);
  Wire.write(0x40);
  Wire.write(0x0);
  Wire.endTransmission();
}

boolean checkReady(){
    byte value;
    Wire.beginTransmission(NT3H_I2C_ADDR);
    Wire.write(0xFE);
    Wire.write(0x06);
    Wire.endTransmission();
    Wire.requestFrom(NT3H_I2C_ADDR,1);
    if (Wire.available() == 1) {
      value=Wire.read();
    }
    return value & 0b10000;
}

void WriteACK(uint8_t *dataBuffer)
{
    setI2CtoNFC();
    // uint8_t dataBuffer[16] = {0};
    dataBuffer[SRAM_SIZE-4] = 'A';
    dataBuffer[SRAM_SIZE-3] = 'C';
    dataBuffer[SRAM_SIZE-2] = 'K';

    Wire.beginTransmission(NT3H_I2C_ADDR);
    Wire.write(0xfb);
    for (size_t i = 0; i < 16; i++)
    {
        Wire.write(dataBuffer[i+3*16]);
    }
    uint8_t error = Wire.endTransmission();

    if (error == 0) {
      Serial.print("ACK successful to register 0x");
      Serial.println(0xfb,HEX);
    } else {
```

```
      Serial.print("ACK fail to register 0x");
      Serial.println(0xfb,HEX);
      Serial.println(error);
    }
    setNFCtoI2C();
}

void ReadDataBlock(const byte block_address, uint8_t *out
{
      Wire.beginTransmission(NT3H_I2C_ADDR);
      Wire.write(block_address);
      Wire.endTransmission();
      Wire.requestFrom(NT3H_I2C_ADDR, out_buffer_length);
      if (Wire.available() == out_buffer_length) {
        for (int i = 0; i < out_buffer_length; i++)
        {
          out_buffer[i+16*(block_address-0xf8)] = Wire.re
          haveWriten ++;
        }
      }
}

void readPages(uint8_t startPage, uint8_t endPage, uint8_
    for (uint8_t page = startPage; page <= endPage; page+
          ReadDataBlock(page, data64,16);
          Serial.println(page);
    }
}

void checkFP(uint8_t *data){
    uint8_t pg_data[48] = {0};
   if (memcmp(data, pg_data, 48) == 0 && data[SRAM_SIZE-4
   {
      allCount = data[SRAM_SIZE-5] + data[SRAM_SIZE-6] <<
      haveWriten = 0;
      Serial.print("allCount: ");
      Serial.println(allCount);

   }else if (memcmp(data, pg_data, 48) == 0 && data[SRAM_
   {
      stopFlag = true;
   }

}

void setup() {
  Wire.begin();
  Serial.begin(115200);
  Wire.setClock(300000);
}
```

```
void printHex(int data) {
  if (data < 16) {
    Serial.print("0");
  }
  Serial.print(data, HEX);
  Serial.print(" ");
}

void loop() {
      while (!stopFlag){
        startPassthrough();
        if (checkReady()){
          byte data64[64] = {0};
          readPages(0xf8,0xfb, data64);
          for (size_t i = 0; i < 64; i++)
          {
            printHex(data64[i]);
          }
          Serial.println();
          if (haveWriten >= allCount && allCount != 0)
            {
                WriteACK(data64);
                haveWriten = 0;
            }
          checkFP(data64);
        }

      }
      Serial.println("==========================");
}
```

到此 NT3H1101 这颗芯片算是研究完毕了，已经可以实现数据通过RF-> I2C 接口的传递了，为后面传递数据打下基础了。

5. STM32L011D4P6 芯片

这颗是意法半导体的一颗低功耗芯片，供电方式完全是靠 NT3H1101 能量收集提供的，所有也只有 L系列的芯片才能满足项目需求。
我之前也没做过意法半导体软件的开发，所以查询了一些资料，发现 STM32CubeMX 这个工具很好用，开发效率大幅度提升。
这颗芯片在这个项目中的职责是 通过I2C 接口对 NT3H1101 寄存器进行设置，图片数据接收，以及通过spi 协议驱动墨水屏。
那么我需要完成以下工作：

1. 将上面的代码移植到stm32平台
2. 移植墨水屏驱动
3. 整合上述俩点

这里为了快速调试，我依然画了一个PCB用作调试：

这里我加了3个储能电容100uf，用于存储 NT3H1101 的提供的能量， 也可以为驱动墨水屏提供更稳定的电源。用了一个跳线冒，防止反向给LDO供电导致芯片损坏。

关于对NT3H1101寄存器操作代码移植 没什么可说的，在 STM32CubeMX配置好对应的gpio功能即可，时钟什么的默认即可。

这里说一下 对墨水屏驱动的移植，以及整合过程中出现的问题。
墨水屏驱动 用spi协议驱动, 基本上SPI协议驱动都有俩个关键函数，
`sendCommand` `sendData` 其区别在于是否拉高DC线，在拉高时，就是 `sendData` , 拉低时就是 `sendCommand` , 剩下的就是按照 已有的驱动给这俩个函数喂入不同的数据，所以墨水屏驱动移植只需要修改 不同平台的SPI通信这类底层函数的调用，上层喂入数据复制粘贴即可。

6. 图像数据的分析与构建

整合过程中，我需要理解墨水屏的原始数据是怎么表示的，这样才能构建图片数据：
在黑白墨水屏中的数据只用黑白俩种表示，不存在中间值（灰度屏除外），一个字节有8位，每一位都可以表示黑或白。
那么一块 104x212 分辨率的墨水屏，其图像数据可以用 104x212/8 = 2756 字节表示。

对于黑白红 三色墨水屏，也是类似原理，不过数据量需要x2才可以，分别表示在 104x212 空间内 白-黑 白-红 像素

这里我写了一个将图像转换 黑白红 3色的算法，最后保存成墨水屏驱动所需的图像格式：

复制代码 隐藏代码

```
import cv2
import numpy as np

pal_arr = [
    # 黑白调色板（索引0）
    [(0, 0, 0), (255, 255, 255)],
    # 黑红调色板（索引1）- 注意：这里只有黑色和红色
    [(0, 0, 0), (0, 0, 255)],
    # 黑、红、白三色调色板（索引2）
    [(0, 0, 0), (0, 0, 255), (255, 255, 255)]
]

# EPD 显示配置
epd_arr = [
    # 格式：[宽度, 高度, 调色板索引]
    [800, 600, 2],  # 使用三色调色板
]

def get_near(r, g, b, is_red_palette=True):
    """
```

```
    改进的颜色判断函数
    对于红黑调色板，只有当像素与红色相似时才返回红色索引
    """
    if not is_red_palette:
        # 黑白调色板：简单亮度判断
        gray = 0.299 * r + 0.587 * g + 0.114 * b
        return 0 if gray < 128 else 1

    # 对于红黑调色板，进行更复杂的颜色判断

    # 计算亮度
    gray = 0.299 * r + 0.587 * g + 0.114 * b

    # 计算与纯红色的相似度
    red_similarity = r - max(g, b)

    # 计算与纯黑色的相似度
    black_similarity = 255 - gray  # 值越大表示越接近黑色

    # 判断逻辑：
    # - 如果很暗，使用黑色
    # - 如果与红色相似且不是太暗或太亮，使用红色
    # - 否则根据亮度使用黑色或白色

    if gray < 60:  # 很暗的区域
        return 0  # 黑色
    elif gray > 200:  # 很亮的区域
        return 2  # 白色
    elif red_similarity > 30 and 80 < gray < 180:  # 与红色
        return 1  # 红色
    else:
        # 其他情况根据亮度决定
        return 0 if gray < 128 else 2

def add_val(err_arr, r, g, b, factor):
    """
    将误差值乘以系数后加到误差数组中
    """
    factor /= 16.0
    return [
        err_arr[0] + r * factor,
        err_arr[1] + g * factor,
        err_arr[2] + b * factor
    ]

def proc_img(image,x=0, y=0, w=None, h=None):
    """
    处理图像的主函数

    参数:
    - image: 输入图像
```

```
        - is_red: 是否使用红黑调色板（True）或黑白调色板（False）
        - x, y: 处理区域的起始坐标
        - w, h: 处理区域的宽度和高度
        """

        # 获取图像尺寸
        sH, sW = image.shape[:2]

        # 设置默认处理区域
        if w is None:
            w = sW
        if h is None:
            h = sH

        # 选择调色板
        epd_ind = 0  # 使用第一个EPD配置
        pal_ind = epd_arr[epd_ind][2]

        cur_pal = pal_arr[pal_ind]

        # 创建输出图像
        output = np.zeros((h, w, 3), dtype=np.uint8)

        err_arr = [np.zeros((w, 3), dtype=np.float32) for _ i
        a_ind = 0
        b_ind = 1

        for j in range(h):
            y_pos = y + j
            if y_pos < 0 or y_pos >= sH:
                # 超出边界，使用棋盘格填充
                for i in range(w):
                    color_idx = 0 if (i + j) % 2 == 0 else 1
                    output[j, i] = cur_pal[color_idx]
                continue

            # 交换误差数组索引
            a_ind, b_ind = b_ind, a_ind
            # 重置当前行的误差
            err_arr[b_ind] = np.zeros((w, 3), dtype=np.float3

            for i in range(w):
                x_pos = x + i
                if x_pos < 0 or x_pos >= sW:
                    # 超出边界，使用棋盘格填充
                    color_idx = 0 if (i + j) % 2 == 0 else 1
                    output[j, i] = cur_pal[color_idx]
                    continue

                # 获取像素值和当前误差
                pixel = image[y_pos, x_pos]
```

```
                b, g, r = pixel  # OpenCV使用BGR格式
                old_err = err_arr[a_ind][i]

                # 添加误差到像素值
                r_new = r + old_err[0]
                g_new = g + old_err[1]
                b_new = b + old_err[2]

                # 钳制值到0-255范围
                r_new = np.clip(r_new, 0, 255)
                g_new = np.clip(g_new, 0, 255)
                b_new = np.clip(b_new, 0, 255)

                # 计算最接近的颜色索引
                color_idx = get_near(r_new, g_new, b_new)
                color_val = cur_pal[color_idx]

                # 设置输出像素
                output[j, i] = color_val

                # 计算误差
                r_err = r_new - color_val[2]  # 注意：OpenCV是I
                g_err = g_new - color_val[1]
                b_err = b_new - color_val[0]

                # 扩散误差到相邻像素
                if i == 0:
                    # 第一列
                    if i < w - 1:
                        err_arr[b_ind][i] += np.array([r_err,
                        err_arr[b_ind][i+1] += np.array([r_er
                        err_arr[a_ind][i+1] += np.array([r_er
                elif i == w - 1:
                    # 最后一列
                    err_arr[b_ind][i-1] += np.array([r_err, g
                    err_arr[b_ind][i] += np.array([r_err, g_e
                else:
                    # 中间列
                    err_arr[b_ind][i-1] += np.array([r_err, g
                    err_arr[b_ind][i] += np.array([r_err, g_e
                    err_arr[b_ind][i+1] += np.array([r_err, g
                    err_arr[a_ind][i+1] += np.array([r_err, g

    return output

def demo():
    width = 104
    heigh = 212

    width = 128
    heigh = 296
```

```
# width = 128
# heigh = 250
test_image = cv2.imread("/home/wshuo/test.png")
# test_image = cv2.imread("images2.jpg")
# 显示原始图像
height_,width_ = test_image.shape[:2]
print(height_,width_)
if width_ < height_:
    test_image = cv2.resize(test_image,(width,heigh))

else:
    test_image = cv2.resize(test_image,(heigh,width))
    test_image = cv2.transpose(test_image)

cv2.imshow("Original Image", test_image)

# 误差扩散 + 红黑
resultImg = proc_img(test_image)
cv2.imshow("Error Diffusion (Red&Black)", resultImg)

print("按任意键关闭窗口...")
cv2.waitKey(0)
cv2.destroyAllWindows()

R = np.reshape(resultImg,(width*heigh,-1))
red = [0]*(width*heigh//8)
black = [0]*(width*heigh//8)
# print(len(resultImg[2]))
for index,i in enumerate(R):
    p_i = index // 8
    bit_i = 7 - (index % 8)
    # bit_i = index % 8
    mask_y = ~(1 << bit_i) & 0xff  # 用作与 &
    mask_h = 1 << bit_i & 0xff # 用作或 |
    # print(bin(mask_y), bin(mask_h))
    if sum(i) == 765:
        #white 红黑都置为1
        black[p_i] = black[p_i] | mask_h
        red[p_i] = red[p_i] | mask_h
    elif sum(i) == 255:
        #red 红置为0 黑置为1
        black[p_i] = black[p_i] | mask_h
        red[p_i] = red[p_i] & mask_y
    elif sum(i) == 0:
    # black 红置为1 黑置为0
        black[p_i] = black[p_i] & mask_y
        red[p_i] = red[p_i] | mask_h
    else:
        print("error")
```

```
    # R = black
    R = black + red
    data = b"".join([i.to_bytes(1,byteorder='little') for
    with open("img.data", "wb") as f:
        f.write(data)

demo()
```

7. 整合图像接收与图像显示遇到的问题

STM32L011 RAM只有 2KB， 小的可怜，根本无法存储一整张图像数据，所有这里不能采用预分配空间，接收完所有数据再去驱动屏幕显示，只能实时接收，实时写入到墨水屏的显存，数据接收完毕，调用显示函数，进行图像刷新。所以其中也遇到了一些问题。

1. 屏幕初始化时间过长，导致开启passthrogh模式延迟，app传输数据失败：这个也是我在原项目遇到的问题，解决方案尽快在主控mcu上电后，程序尽快进入到开启passthrough部分准备数据接收。对于无用的delay直接进行干掉（也不能都干掉，必要的reset dalay如果弄掉会导致显存没清除完全），其实问题根源还是在于app端，检测到芯片后立刻进行数据传输，而不是提供一个按钮，如果我写一个app的话这个问题可以完全解决。
    2. 对于原项目的驱动电路有一些特例墨水屏无法驱动，测量后发现可能是boost升压达不到驱动电压要求，这个也是无源NFC供电导致的，在使用外部供电可以解决，我也尝试过修改升压电路中的电阻但是效果不是特别好，好在这类屏幕较少，大部分屏幕都可以完美驱动。
    3. 原项目的 swclk, swdio, 俩个引脚被用作 驱动spi墨水屏，导致单片机在运行过程中无法进行调试和烧录，这个就很考验手速了，按下reset后，立刻download新程序，在还没有对这俩个gpio初始化之前完成可以完成程序下载。当然也就无法进行任何调试了，因为调试也是依赖这俩个引脚，还好我测试板子接了个led灯，我通过控制 led灯亮灭判断程序执行到哪一步了：)

8. 完成第一版PCB绘制

经过以上的研究，打样俩个测试板后，终于可以绘制这个pcb了。

原理图：

原理图对比原项目，去除了通过 GPIO 使能供电，增加了一个可以控制 led, 修改了天线。基本上硬件改动不大，主要还是摸索软件过程较为繁琐。

9. 升级硬件

原本我没打算升级，不过在购买芯片时发现NT3H2111 芯片比 NT3H1101 还便宜（搞不懂）, NT3H2111 比NT3H1101多一个新功能fast write, 这个对RF->I2C传输

速度会有质的提升，原来传输一张图像的速度可能需要10多秒，用了fast write后1、2秒完成。而且NT3H2111完全兼容 NT3H1101。

然后STM32L011D4P6 也找到了更便宜但引脚更多的 STM32L011F4P6 (后来知道便宜是因为库存，现在更贵了)
stm32替换后，多出的引脚可以用作调试了。下载程序也不用考验手速了 :)

基于上述俩个芯片改动，重新绘制了PCB

硬件的升级对软件部分核心逻辑基本不用动，只不过需要改动几个引脚映射。

10. 升级硬件后遇到的问题

没想到的是，当我用上fast write后（app端会自动判断芯片型号，选择用标准的write还是fast write），主控mcu处理对接收数据写入墨水屏显存速度达不到要求，在还没有写完的时候 RF->I2C的 下一份64字节数据就已经到了，导致APP等待读取时间过长从而传输失败。
所以我stm32程序再次进行了修改。

修改前：

复制代码 隐藏代码

```
EPD_SendData(data);

void EPD_SendData(UBYTE Data)
{
    DEV_Digital_Write(EPD_DC_PIN, 1);
    DEV_Digital_Write(EPD_CS_PIN, 0);
    DEV_SPI_WriteByte(Data);
    DEV_Digital_Write(EPD_CS_PIN, 1);
}
```

修改后：

复制代码 隐藏代码

```
DEV_Digital_Write(EPD_DC_PIN, 1);
DEV_Digital_Write(EPD_CS_PIN, 0);
DEV_SPI_Write_nByte(data64, 64);
DEV_Digital_Write(EPD_CS_PIN, 1);
```

简单来说之前一次只传输一个像素点，但是每次都会重复拉高拉低 DC CS(这部分耗时还可以)，主要还是单个SPI传输数据回比一次性传输64字节数据慢很多，修改之后就再没出现过传输失败了。

但是也引入了个新问题：如果最后图片数据不是64的整数倍，会导致写入额外的垃圾数据，对于黑白墨水屏还好，因为额外的数据不会显示出来，但是对于黑白红的3色墨水屏，黑色传输完后，不是64整数倍，会导致额外的红色数据传输成黑色数据，最终效果是红色像素错位。
所以我进行了这部分的计算，这里就不再贴了。后面我会把全部代码开源。

11. 依然没有结束，子项目的开启

我将做好的设备用滴胶封存，这样替代外壳，因为这玩意不用电。这也是最大优势，但是我将做好的成品发给我朋友时，他们手机NFC失败不到:） 可能是滴胶滴厚了，导致场识别不到（不同手机NFC感应距离也不太相同），当然也有天线设计问题。但是我不想在修改这个板子了，所以考虑替代方案。
做一个板子 与 NT3H2111 进行数据交互，相当于是个图片烧录器，这样即使手机不支持NFC也能玩这个设备了，我选择pn532 小红板：

这个设备感应场很强，即使间隔1-2cm也能识别到设备，但是网上很多资料都是对IC卡的解密，对ntag读写的资料比较少，这部分需要从头摸索。
这个板子有几种模式：

模式	作用
HSU	大部分解密IC卡都是通过此模式完成的，因为不需要主控MCU，直接用个USB-TTL小板即可与这个板子进行通信， 通过串口发送数据帧完成对目标卡的读写
I2C	需要主控MCU, 与这块板子进行I2C连接，发送数据帧实现对目标卡的读写
SPI	与I2C相同，只不过使用的协议是I2C

3种模式可以通过板子上的拨码开关进行控制。

主控使用 ESP-12F模组（这玩意买多了，得用上）

目前这部分程序基本已经调通，底板还没到，等到了，经过我验证后把这部分也开源。

12. 开源地址预留占位

stm32L011D4P6版本：https://oshwhub.com/wshuo426/nfc-wu-yuan-mo-shui-ping
stm32L011F6P6版本：https://oshwhub.com/wshuo426/nfc-mo-shui-ping-2

免费评分

参与人数 88	威望 +2	吾爱币 +183	热心值 +79	理由	收起
月夜克星		+ 2	+ 1	我很赞同！	
Voccoo		+ 1	+ 1	用心讨论，共获提升！	
DUJ		+ 1	+ 1	用心讨论，共获提升！	
溯雪		+ 1	+ 1	用心讨论，共获提升！	
qq1269560093		+ 1	+ 1	用心讨论，共获提升！	
MRXZ1994		+ 1	+ 1	用心讨论，共获提升！	